iRAYPLE Industrial Camera

SDK Development Document (C Interface)



V2.3.0

Revision History

No.	Date	Version	Revision Content	Software Version
1	2022/02	V2.0	Optimized some details.	2.3.1
2	2023/10	V2.1	Added the Routine Descriptions of the Multi-device and Device Open Mode.	2.4.1
3	2023/04	V2.2	Added the Routine of the Image Saving.	2.4.2
4	2025/11	V2.3	Added MVPImage Construction Method	2.6.1

Table of Contents

1 Overview **5**
1.1 Download 5
1.2 Compatibility Introduction 5
1.3 Installation Directory 6
1.3.1 SDK for Windows 6
1.3.2 SDK for Linux 7
1.4 Related Resources of SDK (C interface) 7
1.5 SDK Project Configuration 8
1.5.1 VC6.0 8
1.5.1.1 Configure Header File 8
1.5.1.2 Configure lib 9
1.5.2 VS2005 ~ VS2013 11
1.5.2.1 Configure Header File 11
1.5.2.2 Configure lib 12
1.5.3 QT (pro file) 14
1.5.3.1 Copy the Header File 14
1.5.3.2 Edit Pro File 14
1.5.3.3 Runtime Library Configuration 15
2 Overall Call Process of SDK **16**
3 Function Interface Calling of SDK **17**
3.1 System Operation Instruction 17
3.1.1 Obtain Version Information 17
3.1.2 Device Enumeration (Find Device) 17
3.1.3 Handle Creation and Destruction of Device 18
3.2 Camera Operation Instruction 19
3.2.1 Device Open and Close 19
3.2.2 Camera Configuration Download, Storage and Loading 20
3.2.3 Camera Configuration Settings (Only for GigE Camera) 21
3.2.4 Unique Features 23
3.3 Register Event Callback 24
3.4 Camera Stream Resource Operation Instruction 26
3.4.1 Stream Data Configuration 26
3.4.2 Image Collection 28
3.4.3 Acquire ImageData 29
3.4.4 Operations on Streaming and Image Data 31
3.5 Attribute Operation Instruction 33
3.5.1 Determination, Acquisition and Setting of Attribute. 33
3.5.1.1 Propriety Determination 33
3.5.1.2 Attribute Acquisition 34
3.5.1.3 Attribute Setting 38
3.5.2 Execution of Command Attribute 40
3.5.3 Attribute Search and Usage 40

3.5.3.1 Integer Attribute....40
3.5.3.2 Floating Point Attribute....43
3.5.3.3 Boolean Attribute....45
3.5.3.4 Enumeration Attribute....46
3.5.3.5 String Attribute....48
3.5.3.6 Command Attribute....49
3.6 Image Operation....49
3.6.1 Video Recording....49
3.6.2 Image Operation....50
4 Image Format of 3rd-party Platform Conversion....53
4.1 Halcon Object....53
4.1.1 Mono8 (Monochrome Image)....53
4.1.2 Non-Mono8 (Color Image)....53
4.2 OpenCV Object....54
4.2.1 Mono8 (Monochrome Image)....54
4.2.2 Non-Mono8 (Color Image)....54
4.3 QT Object....55
4.3.1 Mono8 (Monochrome Image)....55
4.3.2 Non-Mono8 (Color Image)....55
4.4 MvpImage Object....55
4.4.1 Mono8 (Monochrome Image)....55
4.4.2 Non-Mono8 (Color Image)....56
5 Matched Routine Description....58
6 FAQ (Frequently Asked Question)....60
6.1 Only 8 Images are Obtained by Using the C Interface....60

1 Overview

1.1 Download

You can download the SDK installation package from the official website.

http://download.huaraytech.com/pub/sdk/

Please download the appropriate version of SDK.

Figure 1-1 SDK installation packages



1.2 Compatibility Introduction

Table 1-1 Operating System Requirement

OS Type	Version	Note
Windows *1	Win7 SP1 32-bit/64-bit	Ensure that OS version includes the patch of SP1.
	Win8 32-bit/64-bit	
	Win10 32-bit/64-bit	
Linux (x86) *2	System Version: Ubuntu 12.04 LTS ~ Ubuntu 21.04 LTS (32/64bit)	The upgraded kernel version system is excluded.
	CentOS 6.5 ~ CentOS 8.0 (32/64bit)	The upgraded kernel version system is excluded.

- Windows XP is no longer supported since SDK V2.2.5.
- The supported kernel version of Linux is 2.6.32 to 5.11.0 (Inclusive).

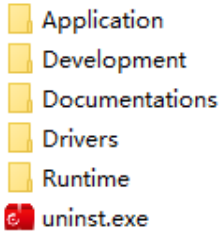
- Use the command of **uname–a** to check the kernel version number.

1.3 Installation Directory

1.3.1 SDK for Windows

The installation directory structure of the SDK for Windows is as follows:

Figure 1-2 Installation directory structure



In V2.1.0, user can select the installation path.

Table 1-2 Installation directory structure description

Application	● **MViewer** is compatible with both 32-bit and 64-bit versions. ● **CamTools** is compatible with both 32-bit and 64-bit versions.
Development	**3rdPartyPlatformAdapter** It includes two plug-in dynamic libraries of third-party algorithm platform (Halcon and Sherlock).
	DOTENT DLL It includes **.Net** library file which is imported when the user creates the project.
	Include It includes the header file of the SDK interface, which explains the basic attributes of various interfaces. The specific usage of header files is in the **samples folder**. The other part is the extension definition of the SDK header file, which defines the SDK error code, the user-defined parameters, the data restriction of some parameters, the enumeration of various behaviors, and the data structure of various information.
	Lib It includes the lib files.

	Samples It includes the various types of demo file, such as C#, Delphi, Lab View, MFC, Python, QT, VB and VC.
Documentations	● **.Net** interface document. ● **ActiveX** interface document. ● **C** interface document. ● **SDK** development document.
Drivers	The installation and uninstallation software of high-performance driver, such as **DShowFilter, GigE, PCIeCL, USB** and **CLVSP.**
Runtime	The compilation dynamic link library when running the SDK. It includes the third-party **GenICam** library which is separated into two folders as 32-bit and 64-bit.

1.3.2 SDK for Linux

The installation directory of SDK for Linux is **/opt/DahuaTech/MVviewer**. The folder structure and introduction are as follows:

Figure 1-3 folder structure description

```
MVviewer
 ├ bin        : GUI Tool, discover/connect camera, image acquisition, properties setting, uninstall script and etc.;
 ├ include    : SDK head files, image convert head files;
 ├ lib        : SDK dll, third-party library, image converting lib;
 ├ module
 |  ├ GigEDriver : GigE camera driver;
 |  └ USBDriver  : USB camera driver;
 └ share
    ├ C       : C samples;
    ├ Qt      : Qt samples;
    └ Python  : Python samples;
```

1.4 Related Resources of SDK (C interface)

Table 1-3 SDK source description

Category	Item	File Storage Folder
SDK Library	Header File	【SDK Installation Directory】 \Development\Include\IMVApi.h \Development\Include\IMVDefines.h
	Lib (32-bit)	【SDK Installation Directory】 \Development\Lib\win32\MVSDKmd.lib
	Lib (64-bit)	【SDK Installation Directory】 \Development\Lib\x64\MVSDKmd.lib

	Runtime Library (32-bit)	【SDK Installation Directory】\Runtime\Win32\MVSDKmd.dll
	Runtime Library (64-bit)	【SDK Installation Directory】\Runtime\x64\MVSDKmd.dll
C API Document	Chinese	【SDK Installation Directory】\Documentations\GenICam_API_C_CHS.chm
	English	【SDK Installation Directory】\Documentations\GenICam_API_C_ENG.chm

1.5 SDK Project Configuration

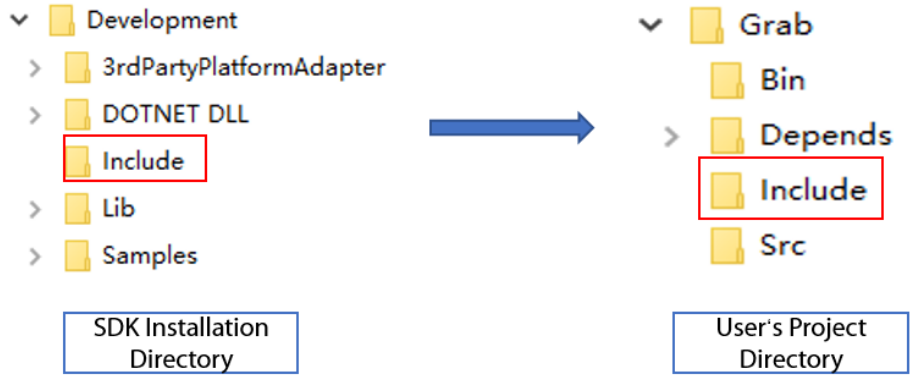
1.5.1 VC6.0

Take **Win32 | Debug** as an example to demonstrate the method of project configuration.

1.5.1.1 Configure Header File

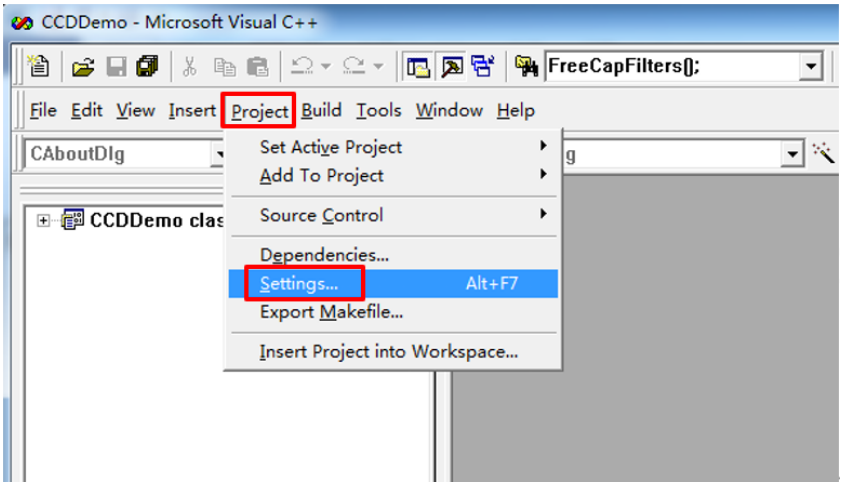
Copy Header File

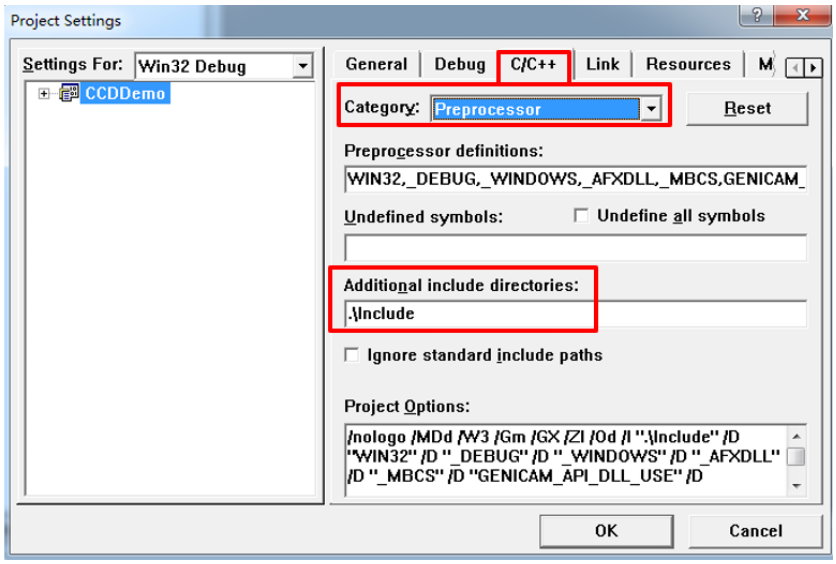
Figure 1-4 Copy header file



Project Configuration

Figure 1-5 Project configuration of header file

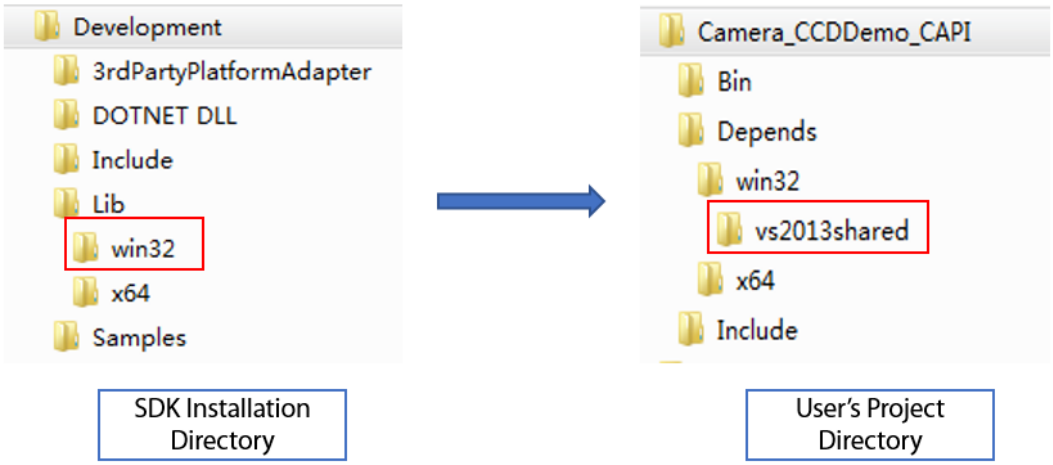




1.5.1.2 Configure lib

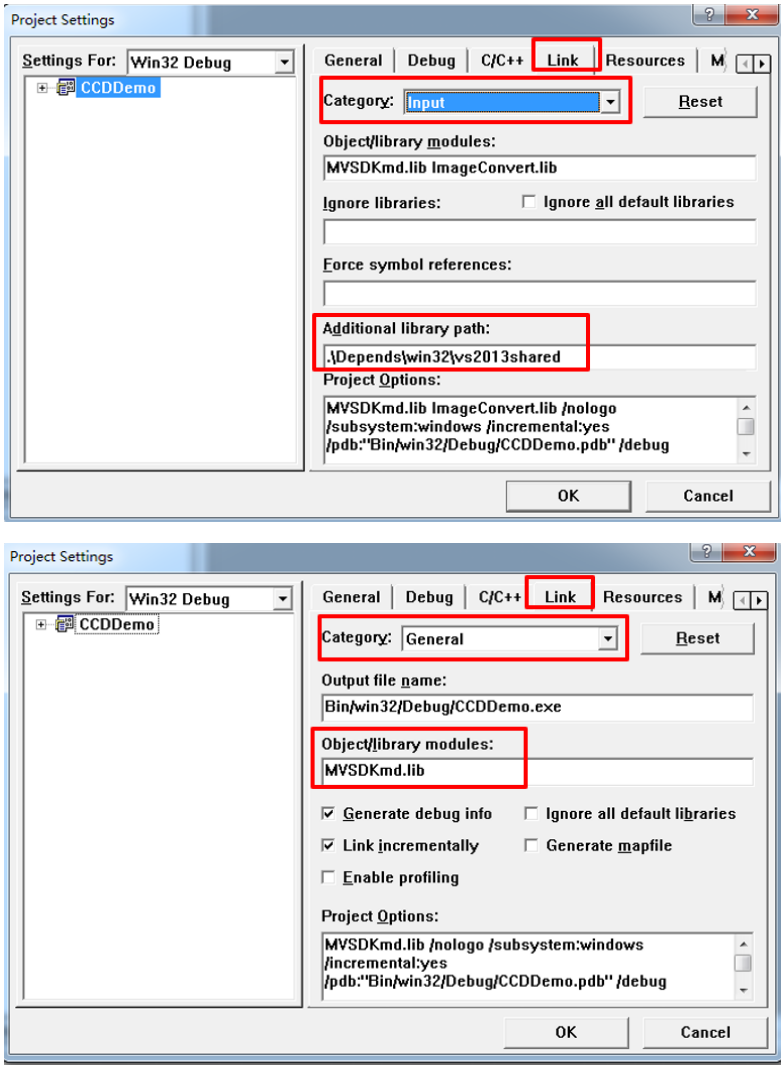
Copy MVSDKmd.lib

Figure 1-6 Copy MVSDKmd.lib



Project Configuration

Figure 1-7 Project configuration of lib



Runtime Library

There is no need to copy the **dll library**. The corresponding **dll library** in the path of 【**SDK Installation Directory**】**\Runtime** will be used by default.

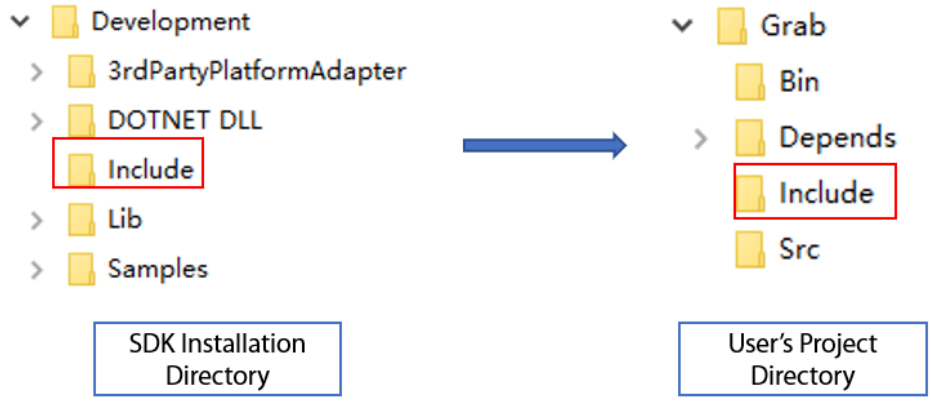
1.5.2 VS2005 ~ VS2013

Take **VS2013 Debug | Win32** as an example to demonstrate the method of project configuration.

1.5.2.1 Configure Header File

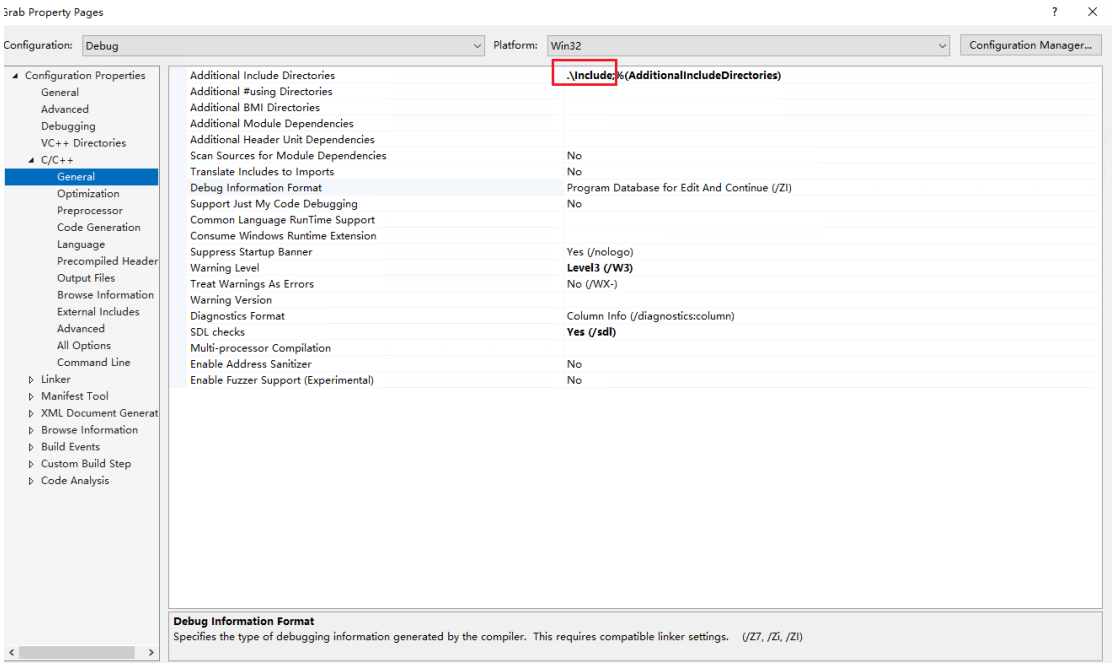
Copy Header File

Figure 1-8 Copy header file



Project Configuration

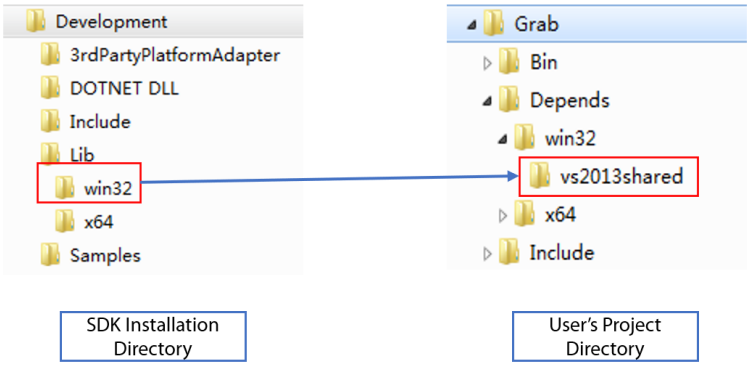
Figure 1-9 Project configuration of header file



1.5.2.2 Configure lib

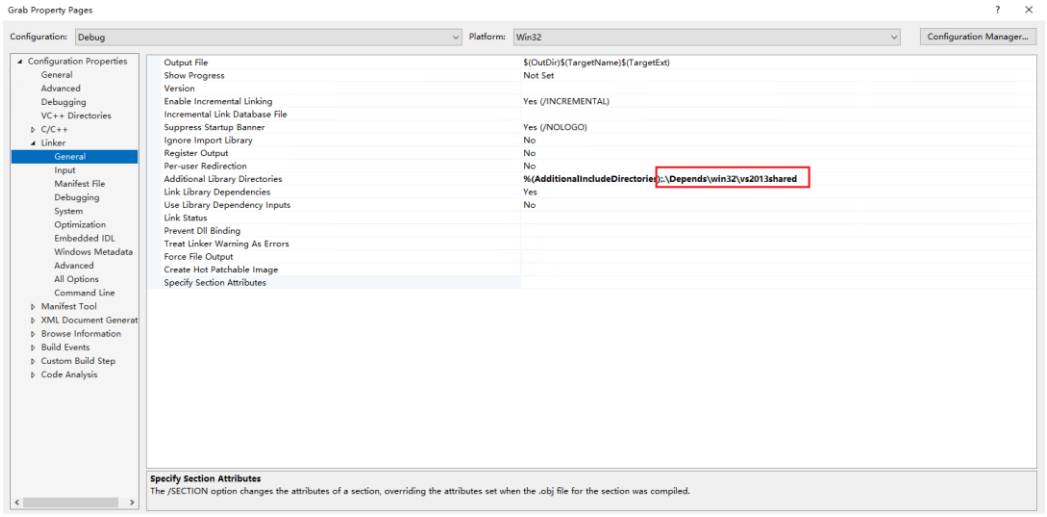
Copy lib

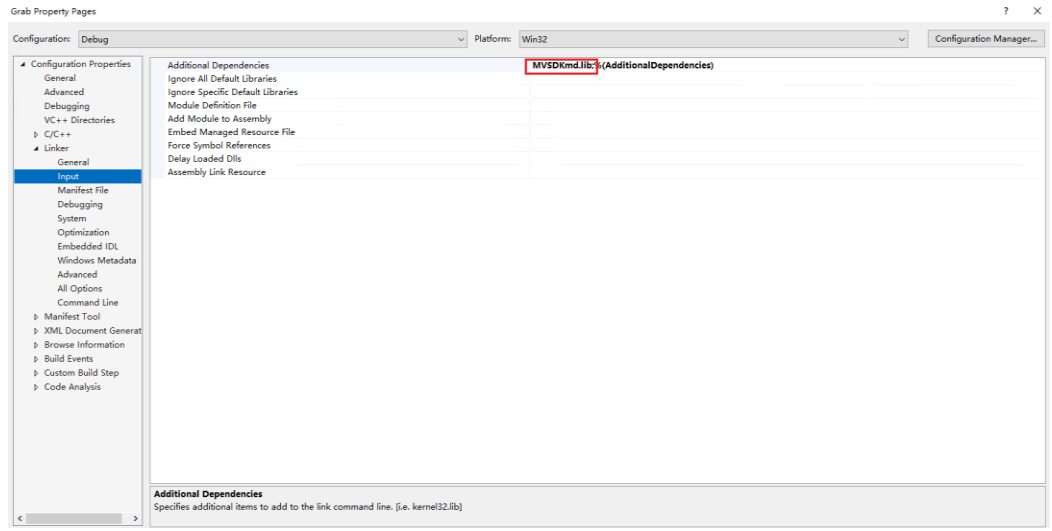
Figure 1-10 Copy lib



Project Configuration

Figure 1-11 Project configuration of lib





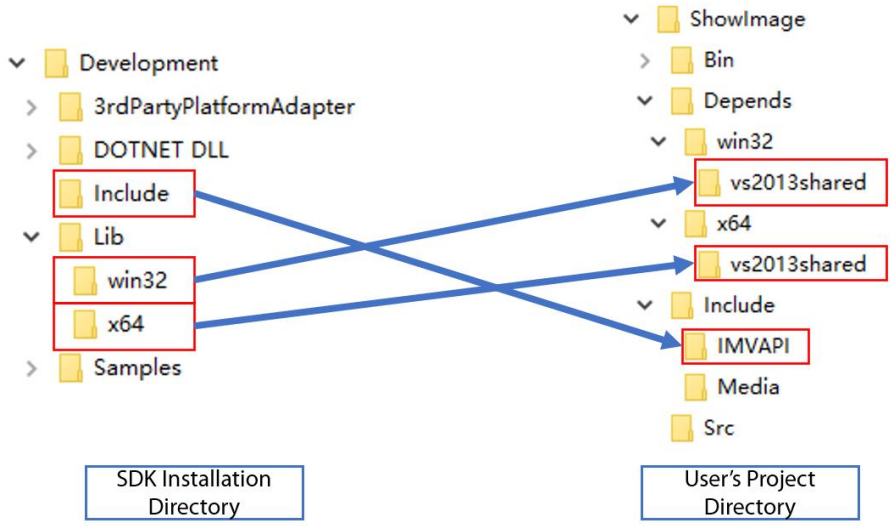
Runtime Library

There is no need to copy the **dll** library. The corresponding **dll** library in the path of 【**SDK Installation Directory**】**\Runtime** will be used by default.

1.5.3 QT (pro file)

1.5.3.1 Copy the Header File

Figure 1-12 Copy header file



1.5.3.2 Edit Pro File

```
Debug {
    contains(QMAKE_COMPILER_DEFINES, _WIN64) {
        LIBS += -L./Depends/x64/vs2013shared                -lMVSDKmd
    }
    else {
        LIBS += -L./Depends/win32/vs2013shared                -lMVSDKmd
    }
}
else {
    contains(QMAKE_COMPILER_DEFINES, _WIN64) {
        LIBS += -L./Depends/x64/vs2013shared                -lMVSDKmd
    }
    else {
        LIBS += -L./Depends/win32/vs2013shared                -lMVSDKmd
      }
}
```

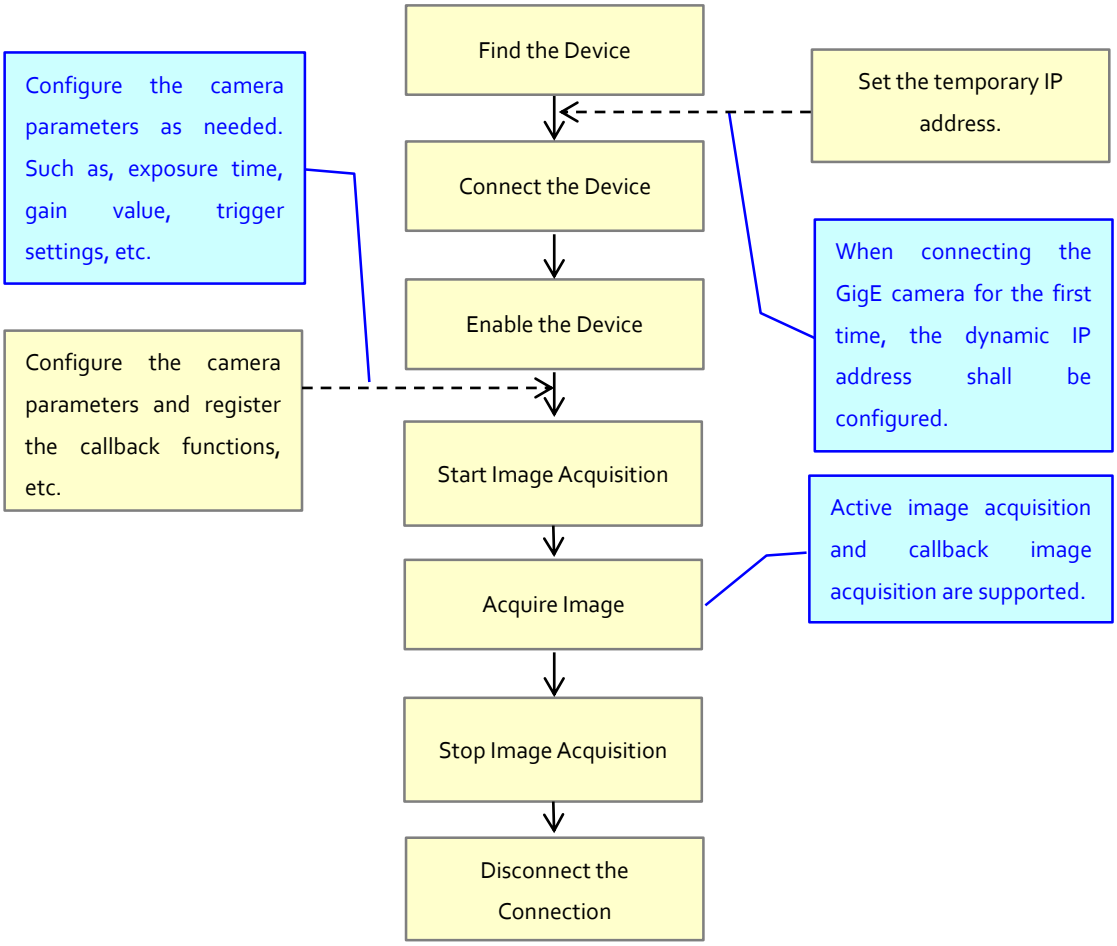
INCLUDEPATH += ./Include

1.5.3.3 Runtime Library Configuration

Please refer to the runtime library configuration of the **Chapter 1.5.2 VS2005 ~ 2013**.

2 Overall Call Process of SDK

Figure 2-1 SDK calling process



3 Function Interface Calling of SDK

The directory of interface header files of SDK: **\Development\Include\IMVApi.h**.

The directory of the macro definition, enumeration, struct and callback function statement of SDK interface: **\Development\Include\IMVDefines.h**.

3.1 System Operation Instruction

3.1.1 Obtain Version Information

- Interface: IMV_API const char* IMV_CALL IMV_GetVersion(void);
  - ◇ Description: Get OS version. If succeeded, return version information. If failed, return **NULL**.
  - ◇ Reference Routine: None.

3.1.2 Device Enumeration (Find Device)

- Interface: IMV_API int IMV_CALL IMV_EnumDevices(OUT IMV_DeviceList *pDeviceList, IN unsigned int interfaceType);
  - ◇ Description: Enumeration Device. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-1 Parameter description

Parameter	Description
pDeviceList [OUT]	Device list.
interfaceType[IN]	The interface type to be enumerated can be combined arbitrarily, such as **interfaceTypeGige | interfaceTypeUsb3**.

  - □ When **interfaceType = interfaceTypeAll,** enumerate all online devices connected through different interfaces.
  - □ When **interfaceType = interfaceTypeGige,** enumerate all online devices connected through the **Gige** interface.
  - □ When **interfaceType = interfaceTypeUsb3,** enumerate all online devices connected through the **USB** interface.
  - □ When **interfaceType = interfaceTypeCL,enumerate** the online devices connected through the **CameraLink** interface.

- The **interfaceType** of the **IMV_EnumDevices** supports any combination of interface types. For example, if you need to enumerate all online devices connected through the GigE port and USB 3.0 port, the **interfaceType** can be written as the **interfaceType = interfaceTypeGige | interfaceTypeUsb3**.S

◇ Reference Routine: All current routines can be referenced.

- Interface: IMV_API int IMV_CALL IMV_EnumDevicesByUnicast(OUT IMV_DeviceList *pDeviceList, IN const char* pIpAddress);

◇ Description: Enumerate devices in unicast format (only for GigE model device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-2 Parameter description

Parameter	Description
pDeviceList [OUT]	Device list.
pIpAddress [IN]	The IP address of the device.

◇ Reference Routine: **UnicastMode**.

3.1.3 Handle Creation and Destruction of Device

- Interface: IMV_API int IMV_CALL IMV_CreateHandle(OUT IMV_HANDLE* handle, IN IMV_ECreateHandleMode mode, IN void* pIdentifier);

◇ Description: Create the handle by specifying the identifier, such as the specified index, device key, custom device name and IP address. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-3 Parameter description

Parameter	Description
Handle [OUT]	Device handle.
mode[IN]	The creation mode of device.
pIdentifier[IN]	Specify the identifier (device key, custom device name, explicitly casting the pointer representing the IP address from **char** type to **void** type; explicitly casting the pointer representing the index from **unsigned int** type to **void** type).

◇ Reference Routine: All current routines can be referenced.

- Interface: IMV_API int IMV_CALL IMV_DestroyHandle(IN IMV_HANDLE handle);

◇ Description: Destroy the handle of device. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-4 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: All current routines can be referenced.

3.2 Camera Operation Instruction

- Interface: IMV_API int IMV_CALL IMV_GetDeviceInfo(IN IMV_HANDLE handle, OUT IMV_DeviceInfo *pDevInfo);
  - ◇ Description: Obtain device information. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-5 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pDevInfo [OUT]	Device information.

  - ◇ Reference Routine: All current routines can be referenced.

3.2.1 Device Open and Close

- Interface: IMV_API int IMV_CALL IMV_Open(IN IMV_HANDLE handle);
  - ◇ Description: Open the device. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-6 Parameter description

Parameter	Description
Handle [IN]	Device handle.



Connect the device according to the handle.

  - ◇ Reference Routine: All current routines can be referenced.
- Interface: IMV_API int IMV_CALL IMV_OpenEx(IN IMV_HANDLE handle, IN IMV_ECameraAccessPermission accessPermission);
  - ◇ Description: Open the device. If succeeded, return **IMV_OK**. If failed, return error code. When using the **IMV_OpenEx** to execute the connection operation, the access rights will be determined.

Table 3-7 Parameter description

Parameter	Description

Handle [IN]	Device handle.
accessPermission[IN]	Control the channel permission (the **accessPermissionControl** is adopted by default in **IMV_Open**).

Connect the device according to the handle and access permission.

◇ Reference Routine: None.

- Interface: IMV_API bool IMV_CALL IMV_IsOpen(IN IMV_HANDLE handle).

◇ Description: Determine whether the device is opened. If opened, return **True**. If close or offline, return **False**.

Table 3-8 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: **ForceIp**.

- Interface: IMV_API int IMV_CALL IMV_Close(IN IMV_HANDLE handle).

◇ Description: Close device. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-9 Parameter description

Parameter	Description
Handle [IN]	Device handle.

Close device according to the handle.

◇ Reference Routine: All current routines can be referenced.

3.2.2 Camera Configuration Download, Storage and Loading

- Interface: IMV_API int IMV_CALL IMV_DownLoadGenICamXML(IN IMV_HANDLE handle, IN const char* pFullFileName);

◇ Description: Download the device description XML file, and save it to the specified path, such as **D:\\xml.zip**. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-10 Parameter description

Parameter	Description

Handle [IN]	Device handle.
pFullFileName[IN]	File saving path.

◇ Reference Routine: None.

- Interface: IMV_API int IMV_CALL IMV_SaveDeviceCfg(IN IMV_HANDLE handle, IN const char* pFullFileName);

  ◇ Description: Saving the device configuration file to the specified folder. The file with the same name that already exists will be overwritten. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-11 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFullFileName [IN]	Full name (including saving path) of the exported device configuration file, such as **D:\\config.xml** or **D:\\config.mvcfg**.

◇ Reference Routine: **LoadAndSaveDeviceCfg**.

- Interface: IMV_API int IMV_CALL IMV_LoadDeviceCfg(IN IMV_HANDLE handle, IN const char* pFullFileName, OUT IMV_ErrorList* pErrorList);

  ◇ Description: Load the xml configuration of device from files. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-12 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFullFileName [IN]	Full name (including saving path) of the device configuration file (XML), such as **D:\\config.xml** or **D:\\config.mvcfg**.
pErrorList [OUT]	The list of attribute name that failed to load. The upper limit of attributes that fails to save and load is **IMV_MAX_ERROR_LIST_NUM**.

◇ Reference Routine: **LoadAndSaveDeviceCfg**.

3.2.3 Camera Configuration Settings (Only for GigE Camera)

- Interface: IMV_API int IMV_CALL IMV_GIGE_ForceIpAddress(IN IMV_HANDLE handle, IN const char* pIpAddress, IN const char* pSubnetMask, IN const char* pGateway);

  ◇ Description: Modify the IP address of device (only for the GigE model device). If

succeeded, return **IMV_OK.** If failed, return error code.

Table 3-13 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pIpAddress [IN]	IP address.
pSubnetMask [IN]	Subnet mask.
pGateway [IN]	Default gateway.

- □ If both **pSubnetMask** and **pGateway** are configured with valid values when calling this function, the valid values shall prevail.
- □ If both **pSubnetMask** and **pGateway** are configured with **NULL** when calling this function, replace it with the subnet mask and gateway of the connected network card during the internal implementation.
- □ If one of the **pSubnetMask** and **pGateway** is configured with **NULL** and the other is **not NULL,** an error will be returned.

◇ Reference Routine: **ForceIp**.

- Interface: IMV_API int IMV_CALL IMV_GIGE_GetAccessPermission(IN IMV_HANDLE handle, OUT IMV_ECameraAccessPermission* pAccessPermission);

◇ Description: Obtain the current access rights of the device (only for GigE model device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-14 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pAccessPermission [OUT]	Current access rights of the device.

◇ Reference Routine: None.

- Interface: IMV_API int IMV_CALL IMV_GIGE_SetAnswerTimeout(IN IMV_HANDLE handle, IN unsigned int timeout);

◇ Description: Configure the timeout period for device's response to SDK command (only for GigE model device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-15 Parameter description

Parameter	Description
Handle [IN]	Device handle.
timeout[IN]	Timeout period (unit: s).

◇ Reference Routine: **GigECommunicationControl**.

3.2.4 Unique Features

- Interface: IMV_API int IMV_CALL IMV_WriteUserPrivateData(IN IMV_HANDLE handle, IN void* pBuffer, IN_OUT unsigned int* pLength);
  - ◇ Description: Write the user-defined data. User can store the 32,768 bytes of user-defined data inside the camera (only for iRAYPLE device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-16 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pBuffer [IN]	Pointer for data buffering.
pLength [IN]	The number of bytes expected to be written.
pLength [OUT]	Actual number of bytes that has been written.

  - ◇ Reference Routine: None.
- Interface: IMV_API int IMV_CALL IMV_ReadUserPrivateData(IN IMV_HANDLE handle, OUT void* pBuffer, IN_OUT unsigned int* pLength);
  - ◇ Description: Read the user-defined data. User can store the 32,768 bytes of user-defined data inside the camera (only for iRAYPLE device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-17 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pBuffer[OUT]	Pointer for data buffering.
pLength [IN]	The number of bytes expected to be written.
pLength [OUT]	Actual number of bytes that has been written.

  - ◇ Reference Routine: None.
- Interface: IMV_API int IMV_CALL IMV_WriteUARTData(IN IMV_HANDLE handle, IN void* pBuffer, IN_OUT unsigned int* pLength);
  - ◇ Description: Write data to the camera serial port register, and history data will be cleared every time user writes (Only for iRAYPLE serial port model device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-18 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pBuffer[IN]	Pointer for data buffering.

pLength [IN]	The number of bytes expected to be written.
pLength [OUT]	Actual number of bytes that has been written.

◇ Reference Routine: None.

- Interface: IMV_API int IMV_CALL IMV_ReadUARTData(IN IMV_HANDLE handle, OUT void* pBuffer, IN_OUT unsigned int* pLength);

  ◇ Description: Read serial port data from the serial register of camera (only for iRAYPLE camera). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-19 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pBuffer[OUT]	Pointer for data buffering.
pLength [IN]	The number of bytes expected to be written.
pLength [OUT]	Actual number of bytes that has been written.

◇ Reference Routine: None.

3.3 Register Event Callback

- Interface: IMV_API int IMV_CALL IMV_SubscribeConnectArg(IN IMV_HANDLE handle, IN IMV_ConnectCallBack proc, IN void* pUser);

  ◇ Description: Device connection status event callback registration. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-20 Parameter description

Parameter	Description
Handle [IN]	Device handle.
proc [IN]	Device connection status event callback function.
pUser [IN]	User-defined data, can be set to NULL.



Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: **ResumeConnect.**

- Interface: IMV_API int IMV_CALL IMV_SubscribeParamUpdateArg(IN IMV_HANDLE handle, IN IMV_ParamUpdateCallBack proc, IN void* pUser);

  ◇ Description: Parameter update event callback registration. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-21 Parameter description

Parameter	Description
Handle [IN]	Device handle.
proc [IN]	Parameter update registration event callback function.
pUser [IN]	User-defined data, can be set to **NULL**.



Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: **Events.**

- Interface: IMV_API int IMV_CALL IMV_SubscribeStreamArg(IN IMV_HANDLE handle, IN IMV_StreamCallBack proc, IN void* pUser);

  ◇ Description: Stream channel event callback registration. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-22 Parameter description

Parameter	Description
Handle [IN]	Device handle.
proc[IN]	Stream channel event callback registration function.
pUser [IN]	User-defined data, can be set to **NULL**.



Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: **Events.**

- Interface: IMV_API int IMV_CALL IMV_SubscribeMsgChannelArg(IN IMV_HANDLE handle, IN IMV_MsgChannelCallBack proc, IN void* pUser);

  ◇ Description: Notification channel event callback registration. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-23 Parameter description

Parameter	Description
Handle [IN]	Device handle.
proc [IN]	Stream channel event callback registration function.
pUser [IN]	User-defined data, can be set to **NULL**.



Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: **Events**.

3.4 Camera Stream Resource Operation Instruction

3.4.1 Stream Data Configuration

- Interface: IMV_API int IMV_CALL IMV_SetBufferCount(IN IMV_HANDLE handle, IN unsigned int nSize);
  - ◇ Description: Configure the buffer quantity of frame data. If succeeded, return **IMV_OK**. If failed, return error code. The configuration cannot be performed during the streaming.

Table 3-24 Parameter description

Parameter	Description
Handle [IN]	Device handle.
nSize [IN]	Buffer quantity.

  - ◇ Reference Routine: None.
- Interface: IMV_API int IMV_CALL IMV_ClearFrameBuffer(IN IMV_HANDLE handle);
  - ◇ Description: Clear the frame data buffer. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-25 Parameter description

Parameter	Description
Handle [IN]	Device handle.

  - ◇ Reference Routine: **ClearFrameBuffer**.
- Interface: IMV_API int IMV_CALL IMV_GIGE_SetInterPacketTimeout(IN IMV_HANDLE handle, IN unsigned int nTimeout);
  - ◇ Description: configure the internal time of data packet resending (only for the GigE model device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-26 Parameter description

Parameter	Description
Handle [IN]	Device handle.
nTimeout [IN]	Interval time of data packet (Unit: ms).



The timeout period of packet loss resending mechanism is under the trigger mode.

◇ Reference Routine: **GigECommunicationControl**.

- Interface: IMV_API int IMV_CALL IMV_GIGE_SetSingleResendMaxPacketNum(IN IMV_HANDLE handle, IN unsigned int maxPacketNum);

  ◇ Description: Configure the internal time of data packet resending (only for the GigE model device). If succeeded, return **IMV_OK**. If failed, return error code. This function is invalid when **maxPacketNum** is 0.

  Table 3-27 Parameter description

Parameter	Description
Handle [IN]	Device handle.
maxPacketNum [IN]	The max number of data packet resending at one time.

  ◇ Reference Routine: **GigECommunicationControl**.

- Interface: IMV_API int IMV_CALL IMV_GIGE_SetMaxLostPacketNum(IN IMV_HANDLE handle, IN unsigned int maxLostPacketNum);

  ◇ Description: Configure the max number of packet loss of the same frame (only for the GigE model device). If succeeded, return **IMV_OK**. If failed, return error code. This function is invalid when **maxLostPacketNum** is 0.

  Table 3-28 Parameter description

Parameter	Description
Handle [IN]	Device handle.
maxLostPacketNum[IN]	The max number of packet loss.



Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

  ◇ Reference Routine: **GigECommunicationControl**.

- Interface: IMV_API int IMV_CALL IMV_USB_SetUrbTransfer(IN IMV_HANDLE handle, IN unsigned int nNum, IN unsigned int nSize);

  ◇ Description: Configure the quantity and volume of data block transmission of the U3V device (only for USB model device). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-29 Parameter description

Parameter	Description
Handle [IN]	Device handle.
nNum[IN]	The number of data block transmission (Range: 5~256).
nSize [IN]	The volume of data block transmission (Range:8~512; Unit: KByte).

□ This interface is valid only when the platform is Linux and no driver is adopted, otherwise, it returns the error code of **IMV_NOT_SUPPORT**.

□ Data block transmission quantity: 5~256 (default value: 64).



If this value is too small, it will affect the transmission efficiency; When multiple cameras are connected and used together, this value can be appropriately reduced.

□ The volume of every data block: 8 KByte ~512 Kbyte (default value: 64 Kbyte).

◇ Reference Routine: None.

3.4.2 Image Collection

- Interface: IMV_API int IMV_CALL IMV_StartGrabbing(IN IMV_HANDLE handle);
  - ◇ Description: Acquiring images. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-30 Parameter description

Parameter	Description
Handle [IN]	Device handle.

  - ◇ Reference Routine: **Grab.**
- Interface: IMV_API int IMV_CALL IMV_StartGrabbingEx(IN IMV_HANDLE handle, IN uint64_t maxImagesGrabbed, IN IMV_EGrabStrategy strategy);
  - ◇ Description: Acquiring images. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-31 Parameter description

Parameter	Description
Handle [IN]	Device handle.

maxImagesGrabbed [IN]	The max number of frames of acquired images. After the specified number of images is acquired, the acquisition will stop. If the specified number is 0, it means that the acquisition is continuously performed.  The parameter of **IMV_StartGrabbing** is 0 by default.
strategy [IN]	Acquisition strategy (**IMV_StartGrabbing** will use the **grabStrartegySequential** to acquire images).

◇ Reference Routine: **GrabStrategy** and **GrabStrategyUpcomingImage**

- Interface: IMV_API bool IMV_CALL IMV_IsGrabbing(IN IMV_HANDLE handle);
  - ◇ Description: Determine whether the device is streaming. If acquiring, return **True**. If not acquiring, return **False**.

Table 3-32 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: **MultipleCamera.**

- Interface: IMV_API int IMV_CALL IMV_StopGrabbing(IN IMV_HANDLE handle);
  - ◇ Description: Stop Streaming. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-33 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: **Grab.**

3.4.3 Acquire ImageData

- Interface: IMV_API int IMV_CALL IMV_AttachGrabbing(IN IMV_HANDLE handle, IN IMV_FrameCallBack proc, IN void* pUser);
  - ◇ Description: Registered frame data callback function (asynchronous acquisition mechanism for frame data). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-34 Parameter description

Parameter	Description

Handle [IN]	Device handle.
proc [IN]	Frame data information callback function. Do not handle the time-consuming operations in this function, otherwise, it will affect the real-time performance of subsequent frame data.
pUser [IN]	User-defined data, can be set to **NULL**.

□ The asynchronous acquisition mechanism and synchronous acquisition mechanism are mutually exclusive, therefore, only one of them can be selected in the system for the same device.

□ Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: **ChunkData, Events, FrameClone, FrameTriggerCount, GrabCallback**

- Interface: IMV_API int IMV_CALL IMV_GetFrame(IN IMV_HANDLE handle, OUT IMV_Frame* pFrame, IN unsigned int timeoutMS);

◇ Description: Receive a frame of image (Synchronous acquisition mechanism for frame data). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-35 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFrame [OUT]	The frame data information.
timeoutMS[IN]	The timeout value for obtaining a frame of image. The INFINITE mode means that it will continuously wait until a frame of image is received, otherwise, it will stop acquiring (unit: ms).

□ This interface does not support the multi-thread callings. The synchronous and asynchronous acquisition mechanism (**IMV_AttachGrabbing**) are mutually exclusive, therefore, only one of them can be selected in the system for the same device.

□ Acquiring images through releasing the internal memory. Use **IMV_ReleaseFrame** to release the image buffer space.

◇ Reference Routine: **ClearFrameBuffer** and **Grab.**

- Interface: IMV_API int IMV_CALL IMV_ReleaseFrame(IN IMV_HANDLE handle, IN IMV_Frame* pFrame);

◇ Description: Release the image buffer memory. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-36 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFrame [IN]	The frame data information.

◇ Reference Routine: **ClearFrameBuffer** and **Grab**.

3.4.4 Operations on Streaming and Image Data

- Interface: IMV_API int IMV_CALL IMV_CloneFrame(IN IMV_HANDLE handle, IN IMV_Frame* pFrame, OUT IMV_Frame* pCloneFrame);
  - ◇ Description: Frame data deep copy and clone. If succeeded, return **IMV_OK**. If failed, return error code. Use **IMV_ReleaseFrame** to release the image buffer space.

Table 3-37 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFrame [IN]	Clone the source frame data information.
pCloneFrame [OUT]	New frame data information.

  - ◇ Reference Routine: **FrameClone**.
- Interface: IMV_API int IMV_CALL IMV_GetChunkDataByIndex(IN IMV_HANDLE handle, IN IMV_Frame* pFrame, IN unsigned int index, OUT IMV_ChunkDataInfo *pChunkDataInfo);
  - ◇ Description: Obtain Chunk Data (only for GigE/USB model cameras). If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-38 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFrame [IN]	The frame data information.
index [IN]	Index ID.
pChunkDataInfo [OUT]	Chunk Data Information.

  - ◇ Reference Routine: **ChunkData**
- Interface: IMV_API int IMV_CALL IMV_GetStatisticsInfo(IN IMV_HANDLE handle, OUT IMV_StreamStatisticsInfo* pStreamStatsInfo);
  - ◇ Description: Obtain statistics information of image acquisition after performing

IMV_StartGrabbing to call the obtaining command. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-39 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pStreamStatsInfo [OUT]	Stream statistics information.

◇ Reference Routine: **StreamStatistics**

- Interface: IMV_API int IMV_CALL IMV_ResetStatisticsInfo(IN IMV_HANDLE handle);

◇ Description: Reset statistics information of image acquisition after performing **IMV_StartGrabbing** to call the resetting command. If succeeded, return **IMV_OK.** If failed, return error code.

Table 3-40 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: **StreamStatistics.**

3.5 Attribute Operation Instruction

3.5.1 Determination, Acquisition and Setting of Attribute.

3.5.1.1 Propriety Determination

- Interface: IMV_API bool IMV_CALL IMV_FeatureIsAvailable(IN IMV_HANDLE handle, IN const char* pFeatureName);
  - ◇ Description: Determine whether the attribute is available. If available, return **True**. If unavailable, return **False**.

Table 3-41 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

  - ◇ Reference Routine: None.
- Interface: IMV_API bool IMV_CALL IMV_FeatureIsReadable (IN IMV_HANDLE handle, IN const char* pFeatureName);
  - ◇ Description: Determine whether the attribute is readable. If readable, return True. If unreadable, return False.

Table 3-42 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

  - ◇ Reference Routine: None.
- Interface: IMV_API bool IMV_CALL IMV_FeatureIsWriteable (IN IMV_HANDLE handle, IN const char* pFeatureName);
  - ◇ Description: Determine whether the attribute is writable. If writable, return **True**. If unwritable, return **False**.

Table 3-43 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

  - ◇ Reference Routine: None.
- Interface: IMV_API bool IMV_CALL IMV_FeatureIsStreamable (IN IMV_HANDLE handle, IN const char* pFeatureName);

◇ Description: Determine whether the attribute is acquirable. If acquirable, return **True**. If unacquirable, return **False**.

Table 3-44 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

◇ Reference Routine: None.

- Interface: IMV_API bool IMV_CALL IMV_FeatureIsValid (IN IMV_HANDLE handle, IN const char* pFeatureName);

◇ Description: Determine whether the attribute is valid. If valid, return True. If invalid, return False.

Table 3-45 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

◇ Reference Routine: **Record.**

3.5.1.2 Attribute Acquisition

- Interface: IMV_API int IMV_CALL IMV_GetIntFeatureValue(IN IMV_HANDLE handle, IN const char* pFeatureName, OUT int64_t* pIntValue);

◇ Description: Obtain the integer attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-46 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The attribute value of Integer.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_GetIntFeatureMin (IN IMV_HANDLE handle, IN const char* pFeatureName, OUT int64_t* pIntValue);

◇ Description: Obtain the configurable min value of the integer attribute. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-47 Parameter description

Parameter	Description
Handle [IN]	Device handle.

pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The min value that the integer can be configured.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_GetIntFeatureMax (IN IMV_HANDLE handle, IN const char* pFeatureName, OUT int64_t* pIntValue);
  - ◇ Description: Obtain the configurable max value of integer attribute. If succeeded, return **IMV_OK.** If failed, return error code.

Table 3-48 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The configurable max value of the integer attribute.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_GetIntFeatureInc (IN IMV_HANDLE handle, IN const char* pFeatureName, OUT int64_t* pIntValue);
  - ◇ Description: Obtain the step size of the integer attribute. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-49 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The step size of integer attribute.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_GetDoubleFeatureValue (IN IMV_HANDLE handle, IN const char* pFeatureName, OUT double* pDoubleValue);
  - ◇ Description: Obtain the floating-point attribute value. If succeeded, return **IMV_OK.** If failed, return error code.

Table 3-50 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The attribute value of floating point.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_GetDoubleFeatureMin (IN IMV_HANDLE handle, IN const char* pFeatureName, OUT double* pDoubleValue);
  - ◇ Description: Obtain the configurable min value of the floating-point attribute. If

succeeded, return **IMV_OK**. If failed, return error code.

Table 3-51 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The configurable min value of the floating-point attribute.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_GetDoubleFeatureMax (IN IMV_HANDLE handle, IN const char* pFeatureName, OUT double* pDoubleValue);

  ◇ Description: Obtain the configurable max value of floating-point attribute. If succeeded, return **IMV_OK.** If failed, return error code.

Table 3-52 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The configurable max value of the floating-point attribute.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_GetBoolFeatureValue (IN IMV_HANDLE handle, IN const char* pFeatureName, OUT bool* pBoolValue);

  ◇ Description: Obtain the Boolean attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-53 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The attribute value of Boolean.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_GetStringFeatureValue (IN IMV_HANDLE handle, IN const char* pFeatureName, OUT IMV_String* pStringValue);

  ◇ Description: Obtain the string attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-54 Parameter description

Parameter	Description
Handle [IN]	Device handle.

pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The attribute value of String.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_GetEnumFeatureValue (IN IMV_HANDLE handle, IN const char* pFeatureName, OUT uint64_t* pEnumValue);

  ◇ Description: Obtain the enumeration attribute value. If succeeded, return **IMV_OK.** If failed, return error code.

Table 3-55 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pEnumValue [OUT]	The attribute value of enumeration.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_GetEnumFeatureSymbol (IN IMV_HANDLE handle, IN const char* pFeatureName, OUT IMV_String* pEnumValueSymbol);

  ◇ Description: Obtain the enumeration symbol value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-56 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pEnumValue [OUT]	The symbol value of enumeration

◇ Reference Routine: **CommPropAccess**

- Interface: IMV_API int IMV_CALL IMV_GetEnumFeatureEntryNum (IN IMV_HANDLE handle, IN const char* pFeatureName, OUT unsigned int* pEntryNum);

  ◇ Description: Obtain the configurable number of the enumeration attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-57 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pEntryNum [OUT]	The configurable quantity of the enumeration attribute value.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_GetEnumFeatureEntrys (IN IMV_HANDLE

handle, IN const char* pFeatureName, OUT IMV_EnumEntryList* pEnumEntryList);

◇ Description: Obtain the enumeration value list of the enumeration attribute value. If succeeded, return **IMV_OK.** If failed, return error code.

Table 3-58 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pEnumEntryList [OUT]	The list of configurable enumeration value of enumeration attribute.

◇ Reference Routine: **CommPropAccess**.

3.5.1.3 Attribute Setting

- Interface: IMV_API int IMV_CALL IMV_SetIntFeatureValue(IN IMV_HANDLE handle, IN const char* pFeatureName, IN int64_t intValue);

  ◇ Description: Obtain the integer attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

  Table 3-59 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
intValue [IN]	The integer attribute value that is to be configured.

  ◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_SetDoubleFeatureValue (IN IMV_HANDLE handle, IN const char* pFeatureName, IN double doubleValue);

  ◇ Description: Configure the floating-point attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

  Table 3-60 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
doubleValue [IN]	Floating point attribute value that is to be configured.

  ◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_SetBoolFeatureValue (IN IMV_HANDLE handle, IN const char* pFeatureName, IN bool boolValue);

  ◇ Description: Configure the **Boolean** attribute value. If succeeded, return

IMV_OK. If failed, return error code.

Table 3-61 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
boolValue [IN]	Boolean attribute value that is to be configured.

◇ Reference Routine: **ChunkData**.

- Interface: IMV_API int IMV_CALL IMV_SetEnumFeatureValue (IN IMV_HANDLE handle, IN const char* pFeatureName, IN uint64_t enumValue);

  ◇ Description: Configure the enumeration attribute value. If succeeded, return **IMV_OK.** If failed, return error code.

Table 3-62 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
enumValue [IN]	Enumeration attribute value that is to be configured.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_API int IMV_CALL IMV_SetEnumFeatureSymbol (IN IMV_HANDLE handle, IN const char* pFeatureName, IN const char* pEnumSymbol);

  ◇ Description: Configure the enumeration symbol value. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-63 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
boolValue [IN]	The symbol value of enumeration that is to be configured.

◇ Reference Routine: **ChunkData**, **CommPropAccess**, **SoftTrigger** and **UserSetControl.**

- Interface: IMV_API int IMV_CALL IMV_SetStringFeatureValue (IN IMV_HANDLE handle, IN const char* pFeatureName, IN const char* pStringValue);

  ◇ Description: Configure the string attribute value. If succeeded, return **IMV_OK.** If failed, return error code.

Table 3-64 Parameter description

Parameter	Description
Handle [IN]	Device handle.

pFeatureName [IN]	Attribute name.
pStringValue [IN]	String attribute value that is to be configured.

◇ Reference Routine: **CommPropAccess**.

3.5.2 Execution of Command Attribute

- Interface: IMV_API int IMV_CALL IMV_ExecuteCommandFeature(IN IMV_HANDLE handle, IN const char* pFeatureName);
  - ◇ Description: Execute the command attribute. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-65 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

  - ◇ Reference Routine: **ChunkData, ClearFrameBuffer, SoftTrigger and UserSetControl.**

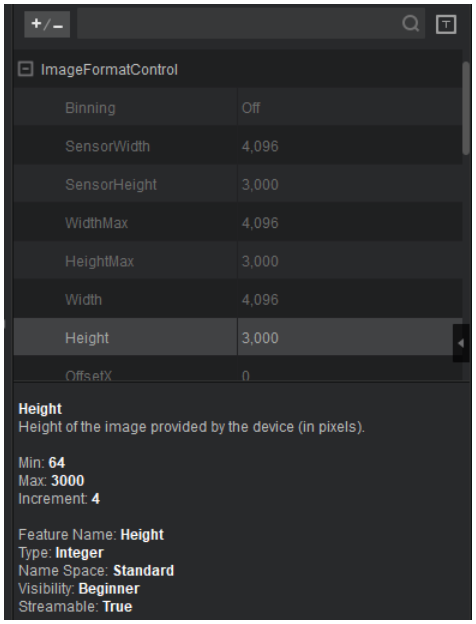
3.5.3 Attribute Search and Usage

User can search the attribute on the client, and learn the detailed information of the attribute, such as data type, value range, etc.

3.5.3.1 Integer Attribute

If user needs to obtain the pixel height information of the camera, find and click the height attribute. The attribute information of it will be displayed in the below. The data type of the height attribute is integer as shown in the figure.

Figure 3-1 Attributes of height



User can obtain and modify the found attributes.

```
    int ret = IMV_OK;
    int64_t heightValue = 0;
    int64_t heightMinValue = 0;
    int64_t heightMaxValue = 0;
    int64_t incrementValue = 0;

    // Obtain attribute value.
    ret = IMV_GetIntFeatureValue(devHandle, "Height", &heightValue);
    if (IMV_OK != ret)
    {
        printf("Get feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Obtain the configurable min value of attribute. If there is no min value, return the error code.
    ret = IMV_GetIntFeatureMin(devHandle, "Height", &heightMinValue);
    if (IMV_OK != ret)
    {
        printf("Get feature minimum value failed! ErrorCode[%d]\n", ret);
        return ret;
    }
```

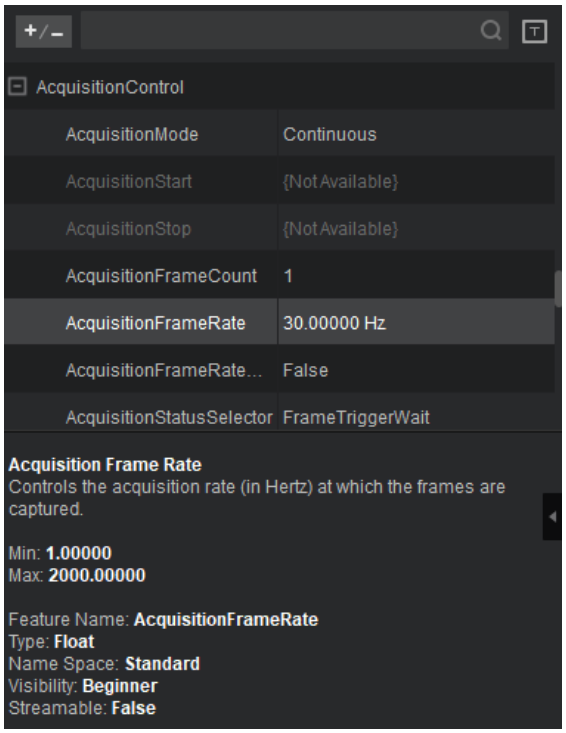
```
    // Obtain the configurable max value of attribute.If there is no max value, return the
error code.
    ret = IMV_GetIntFeatureMax(devHandle, "Height", &heightMaxValue);
    if (IMV_OK != ret)
    {
        printf("Get feature maximum value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Obtain the attribute step size. If there is no step-size value, return the error code.
    ret = IMV_GetIntFeatureInc(devHandle, "Height", &incrementValue);
    if (IMV_OK != ret)
    {
        printf("Get feature increment value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Configure attribute value.
    ret = IMV_SetIntFeatureValue(devHandle, "Height", heightValue);
    if (IMV_OK != ret)
    {
        printf("Set feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }
```

3.5.3.2 Floating Point Attribute

Figure 3-2 Attributes of floating point




```
    int ret = IMV_OK;
    float framerate = 0;
    float frameratemin = 0;
    float frameratemax = 0;

    // Obtain attribute value.
    ret = IMV_GetDoubleFeatureValue (devHandle, "AcquisitionFrameRate", &framerate);
    if (IMV_OK != ret)
    {
        printf("Get feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }
    // Obtain the configurable min value of attribute. If there is no min value, return the error code.
    ret = IMV_GetDoubleFeatureMin(devHandle, "AcquisitionFrameRate", &frameratemin);
    if (IMV_OK != ret)
    {
```

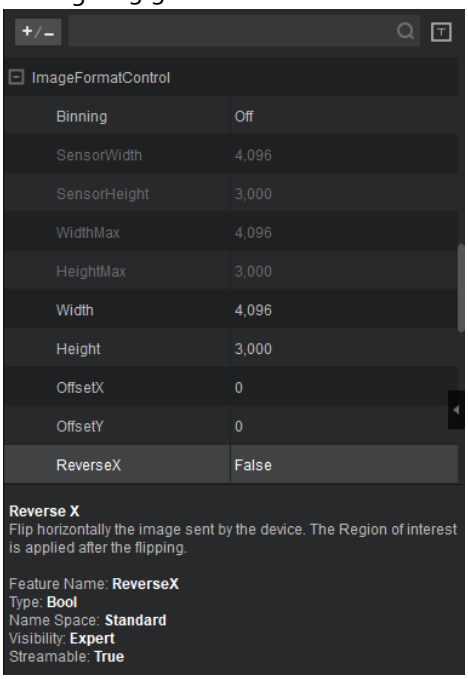
```
        printf("Get feature minimum value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Obtain the configurable max value of attribute.If there is no max value, return the
error code.
    ret = IMV_GetDoubleFeatureMax(devHandle, "AcquisitionFrameRate",
&frameratemax);
    if (IMV_OK != ret)
    {
        printf("Get feature maximum value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Configure attribute value.
    ret = IMV_SetDoubleFeatureValue(devHandle, "AcquisitionFrameRate", framerate);
    if (IMV_OK != ret)
    {
        printf("Set feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }
```

3.5.3.3 Boolean Attribute

Figure 3-3 Attributes of Boolean



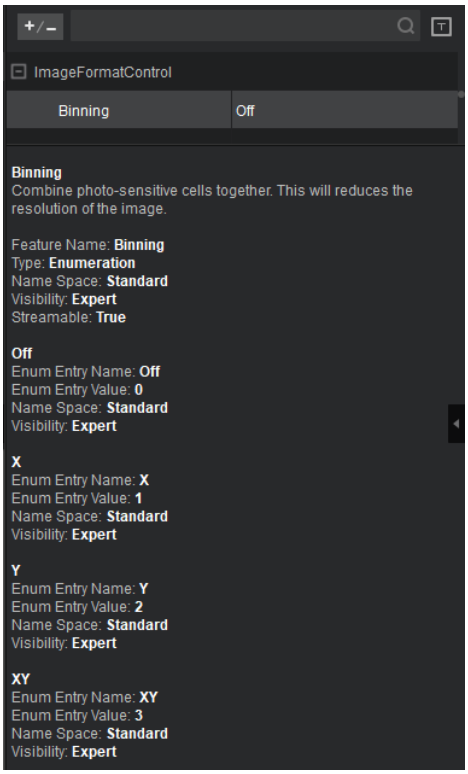

```
int ret = IMV_OK;
    bool reversevalue = 0;

    // Obtain attribute value.
    ret = IMV_GetBoolFeatureValue (devHandle, "ReverseX", &reversevalue);
    if (IMV_OK != ret)
    {
        printf("Get feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Configure attribute value.
    ret = IMV_SetBoolFeatureValue(devHandle, "ReverseX", reversevalue);
    if (IMV_OK != ret)
    {
        printf("Set feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }
```

3.5.3.4 Enumeration Attribute

Figure 3-4 Attributes of enumeration




```
int ret = IMV_OK;
    uint64_t enumValue;
IMV_String pEnumSymbol;
unsigned int pEntryNum;
IMV_EnumEntryList pEnumEntryList;

    // Obtain the configurable enumeration value list of attributes.
    ret = IMV_GetEnumFeatureValue(devHandle, "Binning", &pEnumEntryList);
    if (IMV_OK != ret)
    {
        printf("Get feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    //Obtain the number of configurable enumeration values of attribute
    ret = IMV_GetEnumFeatureEntryNum(devHandle, "Binning", &pEntryNum);
    if (IMV_OK != ret)
```

```
    {
        printf("Get feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Obtain the attribute symbol value.
    ret = IMV_GetEnumFeatureSymbol(devHandle, "Binning", &pEnumSymbol);
    if (IMV_OK != ret)
    {
        printf("Get feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Obtain attribute value.
    ret = IMV_GetIntFeatureInc(devHandle, "Binning", &enumValue);
    if (IMV_OK != ret)
    {
        printf("Get feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

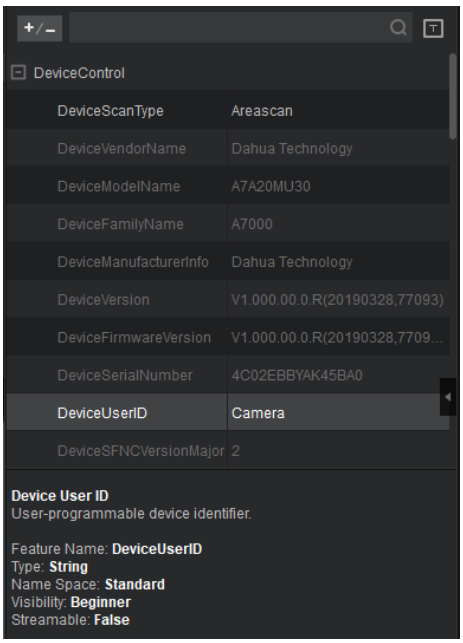
    // Configure attribute symbol value.
    ret = IMV_SetEnumFeatureValue(devHandle, "Binning", pEnumSymbol);
    if (IMV_OK != ret)
    {
        printf("Set feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

// Configure attribute value.
    ret = IMV_SetEnumFeatureValue(devHandle, "Binning", enumValue);
    if (IMV_OK != ret)
    {
        printf("Set feature value failed! ErrorCode[%d]\n", ret);
        return ret;
```

```
}
```

3.5.3.5 String Attribute

Figure 3-5 Attributes of String



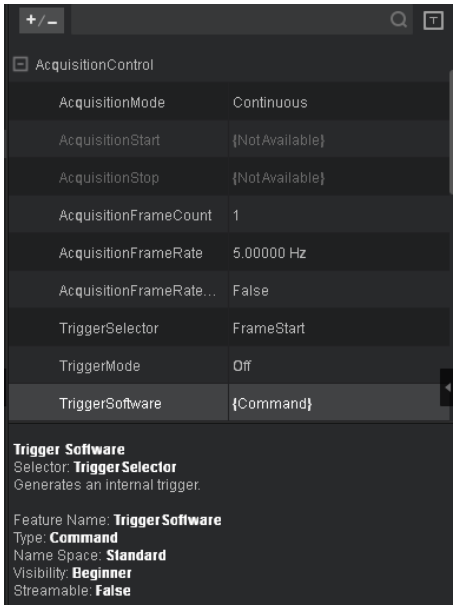

```
int ret = IMV_OK;
    string id = "";

    // Obtain attribute value.
    ret = IMV_GetStringFeatureValue (devHandle, "DeviceUserID", &id);
    if (IMV_OK != ret)
    {
        printf("Get feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Configure attribute value.
    ret = IMV_SetStringFeatureValue(devHandle, "DeviceUserID", id);
    if (IMV_OK != ret)
    {
        printf("Set feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }
```

3.5.3.6 Command Attribute

Figure 3-6 Attributes of command




```
int ret = IMV_OK;
// Execute the attribute value of command.
    ret = IMV_ExecuteCommandFeature (devHandle, "TriggerSoftware");
    if (IMV_OK != ret)
    {
        printf("Execute Command feature failed! ErrorCode[%d]\n", ret);
        return ret;
    }
```

3.6 Image Operation

3.6.1 Video Recording

- Interface: IMV_API int IMV_CALL IMV_OpenRecord(IN IMV_HANDLE handle, IN IMV_RecordParam *pstRecordParam);
  - ◇ Description: Start recording. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-66 Parameter description

Parameter	Description
Handle [IN]	Device handle.

pstRecordParam[IN]	Recording Parameter Struct

◇ Reference Routine: **Record**

- Interface: IMV_API int IMV_CALL IMV_InputOneFrame(IN IMV_HANDLE handle, IN IMV_RecordFrameInfoParam *pstRecordFrameInfoParam);

  ◇ Description: Record a frame of image. If succeeded, return **IMV_OK**. If failed, return error code.

  Table 3-67 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pstRecordFrameInfoParam [IN]	Frame information structure for recording.

  ◇ Reference Routine: None.

- Interface: IMV_API int IMV_CALL IMV_CloseRecord(IN IMV_HANDLE handle);

  ◇ Description: Stop Recording. If succeeded, return **IMV_OK**. If failed, return error code.

  Table 3-68 Parameter description

Parameter	Description
Handle [IN]	Device handle.

  ◇ Reference Routine: **Record**

3.6.2 Image Operation

- Interface: IMV_API int IMV_CALL IMV_PixelConvert(IN IMV_HANDLE handle, IN_OUT IMV_PixelConvertParam* pstPixelConvertParam);

  ◇ Description: Pixel format conversion. If succeeded, return **IMV_OK.** If failed, return error code.

  Table 3-69 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pstPixelConvertParam [IN][OUT]	Pixel format conversion parameter structure

  □ The supported target pixel format: **gvspPixelRGB8 /gvspPixelBGR8 /gvspPixelMono8 /gvspPixelBGRA8**

  □ Convert the raw image data into the pixel format required by the user and store in the specified memory by the caller.

- When pixel format is **YUV411Packed**, the image width must be divisible by 4.
- When pixel format is **YUV422Packed**, the image width must be divisible by 2.
- When pixel format is **YUYVPacked**, the image width must be divisible by 2.
- Converted image: Data storage starts from the first row at the top, which is the default storage direction of camera data.

◇ Reference Routine: **ImageConvert** and **SaveImageToBmp.**

- Interface: IMV_API int IMV_CALL IMV_FlipImage(IN IMV_HANDLE handle, IN_OUT IMV_FlipImageParam* pstFlipImageParam);

◇ Description: Flip Image. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-70 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pstFlipImageParam [IN][OUT]	Image flipping parameter structure.

- The supported pixel format of the vertical and horizontal image flipping: **gvspPixelRGB8 /gvspPixelBGR8 /gvspPixelMono8**.
- Convert the raw image data and store them in the specified memory by the caller.

◇ Reference Routine: **ImageFlip**

- Interface: IMV_API int IMV_CALL IMV_RotateImage(IN IMV_HANDLE handle, IN_OUT IMV_RotateImageParam* pstRotateImageParam);

◇ Description: Clockwise rotation of the image. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-71 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pstRotateImageParam[IN][OUT]	Image rotation parameter structure.

- The supported image format of the 90/180/270 degrees clockwise rotation: **gvspPixelRGB8 / gvspPixelBGR8 / gvspPixelMono8**.
- Rotate the raw image data and store them in the specified memory by the caller.

◇ Reference Routine: **ImageRotate**

- Interface: IMV_API int IMV_CALL IMV_SaveImageToFile(IN IMV_HANDLE handle,

IN IMV_SaveImageToFileParam* pstSaveImageToFileParam);

◇ Description: Save the image. If succeeded, return **IMV_OK**. If failed, return error code.

Table 3-72 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pstSaveImageToFileParam [IN]	Image file saving parameter structure.

□ The supported image format: BMP/JPEG/PNG/TIFF.

□ The supported max height-width size of JPEG format is 65,500.

◇ Reference Routine: **SaveImageToFile.**

4 Image Format of 3rd-party Platform Conversion

4.1 Halcon Object

4.1.1 Mono8 (Monochrome Image)

When image format is **Mono8**, it can be directly converted to Halcon image object.



It is assumed here that the frame is the frame data struct of **IMV_Frame** acquired by SDK.

```
Hobject grabImg; //Halcon image object
gen_image1_extern(&(grabImg),
                  "byte",
                  (Hlong) frame.frameInfo.width,
                  (Hlong) frame.frameInfo.height,
                  (Hlong)( frame.pData ),
                  0);
```

4.1.2 Non-Mono8 (Color Image)

Non-Mono8 format image shall be converted into **BGR** firstly.



- It is assumed here that the frame is the frame data struct of **IMV_Frame** acquired by SDK and the **pBGRbuffer** is the **BGR** data after conversion.
- Please refer to the **Chapter 3.6.2 Image Operation** in details.

```
Hobject grabImg; //Halcon image object
gen_image_interleaved(&grabImg,
                      (Hlong) pBGRbuffer,
                      "bgr", frame.frameInfo.width,
```

```
                frame.frameInfo.height, 0,
                "byte", frame.frameInfo.width,
                frame.frameInfo.height,0,0,8,0);
```

4.2 OpenCV Object

4.2.1 Mono8 (Monochrome Image)

When image format is Mono8, it can be directly converted to **Halcon** image object.



It is assumed here that the frame is the frame data struct of **IMV_Frame** acquired by SDK.

```
//Convert to the object of cv::Mat
cv::Mat image = cv::Mat(frame.frameInfo.height,
                        frame.frameInfo.width,
                        CV_8U,
                            (uint8_t*)(( frame.pData));
```

4.2.2 Non-Mono8 (Color Image)

Non-Mono8 format image shall be converted into **BGR** firstly.



- It is assumed here that the frame is the frame data struct of **IMV_Frame** acquired by SDK and the **pBGRbuffer** is the BGR data after conversion.
- Please refer to the **Chapter 3.6.2 Image Operation** in details.

```
//Convert to the object of cv::Mat
Mat image = cv::Mat(frame.frameInfo.height,
                    frame.frameInfo.width,
                    CV_8UC3,
                    (uint8_t*)pBGRbuffer);
```

4.3 QT Object

4.3.1 Mono8 (Monochrome Image)

When image format is Mono8, it can be directly converted to Halcon image object.



It is assumed here that the frame is the frame data struct of **IMV_Frame** acquired by SDK.

```
QImage image; //Qimage object of Qt
image = QImage(frame.pData,
               frame.frameInfo.width,
               frame.frameInfo.height,
               QImage::Format_Grayscale8);
```

4.3.2 Non-Mono8 (Color Image)

Non-Mono8 format image shall be converted into **BGR** firstly.



- It is assumed here that the frame is the frame data struct of **IMV_Frame** acquired by SDK and the **pBGRbuffer** is the BGR data after conversion.
- Please refer to the **Chapter 3.6.2 Image Operation** in details.

```
QImage image; //Qimage object of Qt
image = QImage((uint8_t*) pBGRbuffer,
               frame.frameInfo.width,
               frame.frameInfo.height,
               QImage::Format_RGB888);
```

4.4 MvpImage Object

4.4.1 Mono8 (Monochrome Image)

For formats such as Mono10 and Mono12, refer to section 3.6.2 for converting the

image format to Mono8. When the camera's image format is Mono8, the captured image data can be directly used to construct the object.



It is assumed here that the **frame** is the frame data struct of **IMV_Frame** acquired by SDK.

```
MvpImage mvpimage;
mvpCreateEmptyImage(&mvpimage);//init
mvpGenImage1(&mvpimage,
                    MVP_IMGDTP_U8,
                    frame.frameInfo.height,
                    frame.frameInfo.width,
                    frame.pData);
mvpReleaseImage(&mvpimage);          //need release
```

4.4.2 Non-Mono8 (Color Image)

Non-Mono8 format image shall be converted into **BGR** firstly.



- It is assumed here that the frame is the frame data struct of **IMV_Frame** acquired by SDK and the **pBGRbuffer** is the BGR data after conversion.
- Please refer to the **Chapter 3.6.2 Image Operation** in details.

```
MvpImage mvpimage;
mvpCreateEmptyImage(&mvpimage);//init
mvpGenImageInterleaved（&mvpimage，
                        pBGRbuffer，
                        MVP_COLOR_SPACE_BGR，
                        frame.frameInfo.width,
                        frame.frameInfo.height,
                        0，
                        MVP_IMGDTP_U8，
                        frame.frameInfo.width,
                        frame.frameInfo.height,
                        0，
                        0，
```

```
                                    -1,
                                    ,0);
    mvpReleaseImage(&mvpimage);          //need release
```

5 Matched Routine Description

C Routine Path: 【SDK Installation Directory】\Development\ Samples\ VC\IMV.

Table 5-1 Routine description

Routine Name	Routine Description
ChunkData	Obtain block data.
ClearFrameBuffer	Clear image buffer.
CommPropAccess	Common attribute routine.
ConnectSpecCamera	Connect the specified device.
DynamicallyLoadDLL	Load the dynamic library.
Events	Event processing mechanism.
ForceIp	Modify the IP address of device (only for the GigE model device).
FrameClone	Clone frame.
FrameTriggerCount	Frame trigger statistic.
GigECommunicationControl	GigE communication control (only for the GigE model device).
Grab	Image capturing in free-run mode (active streaming).
GrabAsynchronous	Asynchronous frame acquisition.
GrabCallback	Streaming callback.
GrabStrategy	Image acquisition strategy.
GrabStrategyUpcomingImage	Acquisition processing.
ImageConvert	Image format conversion.
ImageFlip	Image flipping.
ImageRotate	Clockwise rotation of the image.
LineTrigger	Image acquisition in external trigger mode (callback to acquire).
LoadAndSaveDeviceCfg	Save and load the attribute configuration file of device.
MultiCast	Multi-cast function (only for GigE model device).
MultiCameraGrab	Multi-camera calling routine using C interface in a class-encapsulation.
OpenDeviceMethed	Multi-method of handle creation to open the device routine.
ResumeConnect	Reconnect device after disconnection.

SaveImageToBmp	Save the BMP image.
SaveImageToFile	Save Image to the PC.
SoftTrigger	Acquisition in software trigger mode (callback to acquire).
StreamStatistics	Image acquisition statistics.
UnicastMode	Single-cast function (only for GigE model device).
UserSetControl	Save and load the configuration of device.

6 FAQ (Frequently Asked Question)

6.1 Only 8 Images are Obtained by Using the C Interface

Reason

After acquiring the images using C interface, the images are not released resulting in the buffer space being filled and preventing the acquisition of new images.

Solution

Every time the frame is acquired, the buffer space shall be released by calling the interface of **IMV_ReleaseFrame** as shown in the figure below.

Figure 6-1 IMV_ReleaseFrame calling

```
// Get a frame image
ret = IMV_GetFrame(devHandle, &frame, 1000);
if (IMV_OK != ret)
{
    printf("Get frame failed! ErrorCode[%d]\n", ret);
    return ret;
}
```

```
// Free image buffer
ret = IMV_ReleaseFrame(devHandle, &frame);
if (IMV_OK != ret)
{
    printf("Release frame failed! ErrorCode[%d]\n", ret);
    return ret;
}
```